1) doctor schedule

```
SHIFTS = {1: "Morning", 2: "Afternoon", 3: "Night"}


def is_valid(assignment):
    """Checks whether the current partial or complete assignment satisfies all constraints."""
    d1 = assignment.get("D1")
    d2 = assignment.get("D2")
    d3 = assignment.get("D3")

    # Constraint i: D1 cannot work Night shift
    if d1 == 3:
        return False

    # Constraint iv: D3 cannot work Morning shift
    if d3 == 1:
        return False

    # Constraint ii: D2 must work before D3 (Morning < Afternoon < Night)
    if d2 is not None and d3 is not None:
        if not (d2 < d3):
            return False

    # Constraint iii & v: All assigned shifts must be unique (Only one doctor per shift)
    assigned_values = [v for v in assignment.values() if v is not None]
    if len(assigned_values) != len(set(assigned_values)):
        return False

    return True
```

```
def backtrack(assignment, variables, domains, step_counter):
    # Base Case: All variables are assigned
    if len(assignment) == len(variables):
        print(f"\n[GOAL REACHED] Valid Assignment Found: {assignment}")
        return assignment

    # Select unassigned variable
    unassigned = [v for v in variables if v not in assignment]
    var = unassigned[0]

    for val in domains:
        step_counter[0] += 1
        print(f"Step {step_counter[0]}: Trying {var} = {SHIFTS[val]} ({val})")

        assignment[var] = val

        if is_valid(assignment):
            print(f"  -> Valid partial assignment: {assignment}")
            result = backtrack(assignment, variables, domains, step_counter)
            if result is not None:
                return result
        else:
            print(f"  -> [PRUNED / BACKTRACK] Constraint violation at {assignment}")

        # Undo assignment (Backtrack)
        del assignment[var]

    return None
```

```
if __name__ == "__main__":
    variables = ["D1", "D2", "D3"]
    domains = [1, 2, 3]  # 1: Morning, 2: Afternoon, 3: Night

    print("--- Starting Backtracking Search ---\n")
    step_counter = [0]
    solution = backtrack({}, variables, domains, step_counter)

    print("\n" + "=" * 35)
    print("       FINAL VALID SCHEDULE       ")
    print("=" * 35)
    if solution:
        for doc, shift in solution.items():
            print(f"  {doc} -> {SHIFTS[shift]}")
    else:
        print("No solution found.")
    print("=" * 35)
```

2) star grid

```
import heapq

# 5x5 Grid representation: 0 = Free cell, 1 = Obstacle
```

```
# Grid dimensions
ROWS, COLS = 5, 5

# Start and Goal positions
START = (0, 0)
GOAL = (4, 4)

# Obstacle coordinates
OBSTACLES = {(1, 1), (1, 2), (2, 1), (3, 3)}

# Movement directions: Up, Down, Left, Right
DIRECTIONS = [(-1, 0), (1, 0), (0, -1), (0, 1)]

def manhattan_distance(p1, p2):
    """Calculates Manhattan Distance heuristic h(n)."""
    return abs(p1[0] - p2[0]) + abs(p1[1] - p2[1])

def a_star_search(start, goal):
    """Executes A* search algorithm and prints priority queue updates."""
    # Priority Queue stores tuples: (f_score, counter, current_node)
    # Counter handles tie-breaking in priority queue
    counter = 0
    pq = []

    g_score = {start: 0}
    f_score = {start: manhattan_distance(start, goal)}

    heapq.heappush(pq, (f_score[start], counter, start))
```

```
    parent = {}
    visited = set()

    step = 0
    print("=" * 65)
    print("                A* SEARCH STEP-BY-STEP TRACE                ")
    print("=" * 65)

    while pq:
        current_f, _, current = heapq.heappop(pq)

        if current in visited:
            continue

        visited.add(current)
        step += 1

        print(f"\nStep {step}:")
        print(f"  Popped Node: {current} | g = {g_score[current]}, h =
{manhattan_distance(current, goal)}, f = {current_f}")

        # Check for Goal
        if current == goal:
            print("\n>>> GOAL REACHED! <<<")
            break

        # Explore neighbors
        for dr, dc in DIRECTIONS:
            neighbor = (current[0] + dr, current[1] + dc)
```

```
            r, c = neighbor

            # Check grid bounds and obstacles
            if 0 <= r < ROWS and 0 <= c < COLS and neighbor not in OBSTACLES:
                tentative_g = g_score[current] + 1

                if neighbor not in g_score or tentative_g < g_score[neighbor]:
                    parent[neighbor] = current
                    g_score[neighbor] = tentative_g
                    h = manhattan_distance(neighbor, goal)
                    f = tentative_g + h
                    f_score[neighbor] = f

                    counter += 1
                    heapq.heappush(pq, (f, counter, neighbor))
                    print(f"    Expanded Neighbor {neighbor}: g={tentative_g}, h={h}, f={f}")

    # Reconstruct optimal path
    path = []
    curr = goal
    while curr in parent:
        path.append(curr)
        curr = parent[curr]
    path.append(start)
    path.reverse()

    return path, g_score[goal]

def print_grid_visualization(path):
```

```
    """Displays visual grid in terminal."""
    print("\n" + "=" * 35)
    print("       GRID VISUALIZATION        ")
    print("=" * 35)
    path_set = set(path)
    for r in range(ROWS):
        row_str = ""
        for c in range(COLS):
            pos = (r, c)
            if pos == START:
                row_str += " S "
            elif pos == GOAL:
                row_str += " G "
            elif pos in OBSTACLES:
                row_str += " X "
            elif pos in path_set:
                row_str += " * "
            else:
                row_str += " . "
        print(row_str)
    print("=" * 35)
    print(" Legend: S=Start, G=Goal, X=Obstacle, *=Path, .=Empty\n")

if __name__ == "__main__":
    optimal_path, path_cost = a_star_search(START, GOAL)

    print("\n" + "=" * 65)
    print(f"Optimal Path: {optimal_path}")
    print(f"Total Path Cost: {path_cost}")
```

```
print("=" * 65)


print_grid_visualization(optimal_path)
```

3)online rescue robot

```
import math

# Grid Configuration
GRID_SIZE = 5
START = (0, 0)
GOAL = (4, 4)

# Actual Environment (Unknown to robot initially)
REAL_OBSTACLES = {(1, 1), (1, 2), (2, 1), (3, 3)}
REAL_RISKY_ZONES = {(2, 2), (3, 2)}  # Cost = 3

DIRECTIONS = [(-1, 0), (1, 0), (0, -1), (0, 1)]  # Up, Down, Left, Right

def manhattan_distance(pos):
    """Heuristic h(n): Manhattan distance to Goal."""
    return abs(pos[0] - GOAL[0]) + abs(pos[1] - GOAL[1])

class OnlineRescueRobot:
    def __init__(self, start, goal):
        self.current_pos = start
        self.goal = goal
        self.known_obstacles = set()
        self.known_risky = set()
        # Heuristic lookup table h(s) initialized with Manhattan distance
        self.h_table = {}

    def get_h(self, pos):
        if pos not in self.h_table:
```

```
            self.h_table[pos] = manhattan_distance(pos)
        return self.h_table[pos]

    def sense_adjacent(self):
        """Senses adjacent cells and updates internal map knowledge."""
        r, c = self.current_pos
        for dr, dc in DIRECTIONS:
            adj = (r + dr, c + dc)
            if 0 <= adj[0] < GRID_SIZE and 0 <= adj[1] < GRID_SIZE:
                if adj in REAL_OBSTACLES:
                    self.known_obstacles.add(adj)
                elif adj in REAL_RISKY_ZONES:
                    self.known_risky.add(adj)

    def get_step_cost(self, neighbor):
        """Returns step cost: 3 for risky cells, 1 for normal clear cells."""
        return 3 if neighbor in self.known_risky else 1

    def step(self):
        """Executes one online search step using LRTA* decision rule."""
        self.sense_adjacent()

        best_move = None
        min_f = math.inf

        r, c = self.current_pos
        # Evaluate valid unblocked adjacent cells
        for dr, dc in DIRECTIONS:
            neighbor = (r + dr, c + dc)
```

```
            if 0 <= neighbor[0] < GRID_SIZE and 0 <= neighbor[1] < GRID_SIZE:
                if neighbor not in self.known_obstacles:
                    cost = self.get_step_cost(neighbor)
                    f_val = cost + self.get_h(neighbor)

                    if f_val < min_f:
                        min_f = f_val
                        best_move = neighbor

        # Update local heuristic (Learning step to prevent dead-end loops)
        self.h_table[self.current_pos] = min_f
        # Move robot
        self.current_pos = best_move
        return best_move, self.get_step_cost(best_move)

def run_simulation():
    robot = OnlineRescueRobot(START, GOAL)
    total_cost = 0
    path = [START]

    print("=" * 55)
    print("      ONLINE RESCUE ROBOT NAVIGATION SIMULATION      ")
    print("=" * 55)

    step = 0
    while robot.current_pos != GOAL:
        step += 1
        pos, move_cost = robot.step()
        total_cost += move_cost
```

```
        path.append(pos)
        cell_type = "Risky (Cost=3)" if pos in REAL_RISKY_ZONES else "Clear (Cost=1)"
        print(f"Step {step:2d}: Moved to {pos} | Cell: {cell_type} | Path Cost: {total_cost}")

    print("\n" + "=" * 55)
    print(">>> SURVIVOR LOCATED SUCCESSFULLY! <<<")
    print(f"Optimal Path Taken: {path}")
    print(f"Total Path Cost: {total_cost}")
    print("=" * 55)

if __name__ == "__main__":
    run_simulation()
```